

Claude Certified Architect — Foundations

Complete Study Guide · Based on Official Anthropic Exam Guide

60 Questions

120 Minutes

Pass Score: 720/1000

Multiple Choice

No Guessing Penalty

CONTENTS

1. Exam Overview & Format
2. 6 Exam Scenarios
3. Domain 1: Agentic Architecture & Orchestration (27%)
4. Domain 2: Tool Design & MCP Integration (18%)
5. Domain 3: Claude Code Configuration & Workflows (20%)
6. Domain 4: Prompt Engineering & Structured Output (20%)
7. Domain 5: Context Management & Reliability (15%)
8. 12 Official Sample Questions with Answers
9. 4 Hands-On Preparation Exercises
10. Key Terms & Technology Reference
11. In-Scope vs Out-of-Scope Topics
12. Study Checklist

Exam Overview & Format

WHAT THIS CERT VALIDATES

You can make informed decisions about tradeoffs when implementing real-world solutions with Claude — across Claude Code, Agent SDK, Claude API, and MCP. It tests practical judgment, not just concepts.

Format: 60 multiple-choice questions, each with 1 correct answer and 3 distractors.

Unanswered = incorrect, but there's no penalty for guessing. **Always answer every question.**

Scoring: Scaled 100–1,000. Minimum passing score is **720**. Pass/fail designation.

Scenarios: 4 of the 6 scenarios below are presented at random. Questions are grounded in those scenarios.

Target candidate: Solution architect with 6+ months hands-on experience building with Claude APIs, Agent SDK, Claude Code, and MCP.

ANTI-PATTERN TRAP

Distractors are designed for candidates with incomplete knowledge. They often sound reasonable. The exam tests whether you know the *most effective* approach — not just a working one.

The 6 Exam Scenarios

4 of these 6 will appear on your exam. Know all 6.

Scenario 1: Customer Support Resolution Agent

Building a support agent with tools: `get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`. Target: 80%+ first-contact resolution. Tests: Domain 1, 2, 5

Scenario 2: Code Generation with Claude Code

Using Claude Code for code gen, refactoring, debugging, documentation with CLAUDE.md, custom slash commands, and plan mode. Tests: Domain 3, 5

Scenario 3: Multi-Agent Research System

Coordinator delegates to specialized subagents: web search, document analysis, synthesis, report generation. Tests: Domain 1, 2, 5

Scenario 4: Developer Productivity with Claude

Agent helps engineers explore codebases, understand legacy systems, generate boilerplate using built-in tools (Read, Write, Bash, Grep, Glob) + MCP. *Tests: Domain 2, 3, 1*

Scenario 5: Claude Code for Continuous Integration

Integrating Claude Code into CI/CD: automated code reviews, test generation, PR feedback. *Tests: Domain 3, 4*

Scenario 6: Structured Data Extraction

Extracting information from unstructured documents using JSON schemas, validating output, handling edge cases. *Tests: Domain 4, 5*

Domain 1: Agentic Architecture & Orchestration

1

Agentic Architecture & Orchestration

27% — Highest weight

1.1 — DESIGN AND IMPLEMENT AGENTIC LOOPS

KNOW

- Agentic loop lifecycle: send request → inspect `stop_reason` → execute tool → append result → repeat
- `stop_reason: "tool_use"` → continue loop; `stop_reason: "end_turn"` → present final response
- Model-driven decision-making vs pre-configured tool sequences

DO

- Loop continues when `stop_reason == "tool_use"`, terminates on `"end_turn"`
- Always append tool results to conversation context between iterations

ANTI-PATTERNS TO AVOID

Parsing natural language signals to determine loop termination · Setting arbitrary iteration caps as primary stopping mechanism · Checking assistant text content as completion indicator

1.2 — ORCHESTRATE MULTI-AGENT SYSTEMS (COORDINATOR-SUBAGENT)

KNOW

- **Hub-and-spoke:** coordinator manages all inter-subagent communication, error handling, info routing
- Subagents have **isolated context** — they do NOT inherit coordinator's conversation history
- Coordinator role: task decomposition, delegation, result aggregation, routing decisions
- Risk: overly narrow decomposition → incomplete coverage of broad topics

DO

- Design coordinator prompts for research goals + quality criteria (not step-by-step instructions)
- Partition research scope across subagents to minimize duplication
- Route all subagent communication through coordinator for observability
- Implement iterative refinement loops: synthesize → identify gaps → re-delegate → re-synthesize

1.3 — CONFIGURE SUBAGENT INVOCATION, CONTEXT PASSING, AND SPAWNING

KNOW

- **Task** tool is the mechanism for spawning subagents
- **allowedTools** must include **"Task"** for a coordinator to invoke subagents
- Subagent context must be **explicitly provided in the prompt** — no automatic inheritance
- **AgentDefinition**: descriptions, system prompts, tool restrictions per subagent type

DO

- Pass complete prior findings directly in subagent prompt (web results, doc analysis → synthesis agent)
- Use structured data formats to separate content from metadata (source URLs, page numbers)
- **Spawn parallel subagents** by emitting multiple **Task** calls in a single coordinator response

1.4 — MULTI-STEP WORKFLOWS WITH ENFORCEMENT AND HANDOFF PATTERNS

KNOW

- Programmatic enforcement (hooks, prerequisite gates) vs prompt-based guidance

- When deterministic compliance is required (e.g., identity verification before financial ops) → prompt instructions alone have non-zero failure rate
- Structured handoff protocols: customer details, root cause analysis, recommended actions

DO

- Implement programmatic prerequisites that block downstream tools (block `process_refund` until `get_customer` returns verified ID)
- Decompose multi-concern requests into parallel investigations, synthesize unified resolution

1.5 — APPLY AGENT SDK HOOKS FOR TOOL CALL INTERCEPTION

KNOW

- `PostToolUse` hook: intercepts tool results for transformation before model processes them
- Hook patterns for outgoing tool call interception (enforce compliance rules, block refunds above threshold)
- Hooks = deterministic guarantees; prompt instructions = probabilistic compliance

KEY RULE

Use hooks (not prompts) when business rules require guaranteed compliance — e.g., blocking refunds over \$500. Prompt-only approaches have non-zero failure rates for critical operations.

1.6 — TASK DECOMPOSITION STRATEGIES

- Fixed sequential pipelines (prompt chaining) → predictable, multi-aspect reviews
- Dynamic adaptive decomposition → open-ended investigation tasks, generates subtasks based on findings
- Splitting large code reviews: per-file local analysis passes + separate cross-file integration pass

1.7 — SESSION STATE, RESUMPTION, AND FORKING

- `--resume <session-name>` to continue a specific named prior conversation
- `fork_session` to create independent branches from a shared analysis baseline
- Resume when prior context is mostly valid; start fresh with injected summaries when tool results are stale
- After resuming: inform agent of specific file changes for targeted re-analysis

Domain 2: Tool Design & MCP Integration

2 Tool Design & MCP Integration

18%

2.1 — DESIGN EFFECTIVE TOOL INTERFACES

KNOW

- Tool descriptions are the **primary mechanism LLMs use for tool selection**
- Minimal descriptions → unreliable selection among similar tools
- Include: input formats, example queries, edge cases, boundary explanations
- Ambiguous or overlapping descriptions cause misrouting (e.g., `analyze_content` vs `analyze_document`)
- System prompt wording affects tool selection — keyword-sensitive instructions can create unintended associations

DO

- Write descriptions that differentiate purpose, expected inputs, outputs, and when to use vs alternatives
- Rename tools to eliminate functional overlap (e.g., `analyze_content` → `extract_web_results`)
- Split generic tools into purpose-specific ones: `analyze_document` → `extract_data_points` , `summarize_content` , `verify_claim_against_source`

2.2 — STRUCTURED ERROR RESPONSES FOR MCP TOOLS

KNOW

- MCP `isError` flag for communicating tool failures to the agent
- Error categories: **transient** (timeouts), **validation** (invalid input), **business** (policy violations), **permission**
- Generic "Operation failed" prevents the agent from making appropriate recovery decisions
- Retryable vs non-retryable errors — return structured metadata to prevent wasted retries

DO

- Return `errorCategory` (transient/validation/permission), `isRetryable` boolean, human-readable description
- Subagents handle local recovery for transient failures; propagate only unresolvable errors to coordinator

- Distinguish **access failures** (need retry decision) from **valid empty results** (successful query with no matches)

2.3 — TOOL DISTRIBUTION AND TOOL_CHOICE CONFIGURATION

- 18 tools vs 4-5 tools: more tools degrades selection reliability by increasing decision complexity
- Agents with out-of-specialization tools tend to misuse them
- `tool_choice: "auto"` — model may return text; `"any"` — must call a tool; forced: `{"type": "tool", "name": "..."}"`
- Use `tool_choice: "any"` to guarantee tool call instead of conversational text
- Forced selection: ensure a specific tool is called first (e.g., `extract_metadata` before enrichment)

2.4 — INTEGRATE MCP SERVERS INTO CLAUDE CODE AND AGENT WORKFLOWS

- `.mcp.json` (project-level, shared team tooling) vs `~/.claude.json` (user-level, personal/experimental)
- Environment variable expansion: `${GITHUB_TOKEN}` in `.mcp.json` for credential management without committing secrets
- Tools from all configured MCP servers discovered at connection time, available simultaneously
- MCP resources: expose content catalogs (issue summaries, doc hierarchies, database schemas) to reduce exploratory tool calls
- Prefer existing community MCP servers for standard integrations (Jira); reserve custom servers for team-specific workflows

2.5 — BUILT-IN TOOLS (READ, WRITE, EDIT, BASH, GREP, GLOB)

- **Grep:** content search (function names, error messages, import statements)
- **Glob:** file path pattern matching (finding files by name/extension, e.g., `**/*.test.tsx`)
- **Read/Write:** full file operations; **Edit:** targeted modifications using unique text matching
- When Edit fails (non-unique anchor text) → use Read + Write as fallback
- Build codebase understanding incrementally: Grep entry points → Read to follow imports → trace flows

Domain 3: Claude Code Configuration & Workflows

3 Claude Code Configuration & Workflows

20%

3.1 — CONFIGURE CLAUDE.MD FILES

KNOW

- Hierarchy: `~/.claude/CLAUDE.md` (user-level, not shared) → `.claude/CLAUDE.md` or root `CLAUDE.md` (project-level) → subdirectory `CLAUDE.md` (directory-level)
- `@import` syntax for referencing external files to keep CLAUDE.md modular
- `.claude/rules/` directory for topic-specific rule files (alternative to monolithic CLAUDE.md)

DO

- Diagnose config hierarchy issues (e.g., new team member not receiving instructions because they're in user-level config)
- Use `/memory` command to verify which files are loaded and diagnose inconsistent behavior
- Split large CLAUDE.md into focused files in `.claude/rules/` (e.g., `testing.md`, `api-conventions.md`, `deployment.md`)

3.2 — CUSTOM SLASH COMMANDS AND SKILLS

KNOW

- `.claude/commands/` (project-scoped, shared via version control) vs `~/.claude/commands/` (user-scoped, personal)
- Skills in `.claude/skills/` with `SKILL.md` files supporting frontmatter: `context:` `fork`, `allowed-tools`, `argument-hint`
- `context: fork` → runs skill in isolated sub-agent context, prevents skill output from polluting main conversation
- Personal skill variants: create in `~/.claude/skills/` with different names to avoid affecting teammates

DO

- Use `context: fork` to isolate verbose outputs (codebase analysis, brainstorming) from main session
- Configure `allowed-tools` to restrict tool access during skill execution (e.g., limit to file writes only)

- Choose skills for on-demand task-specific workflows; CLAUDE.md for always-loaded universal standards

3.3 — PATH-SPECIFIC RULES FOR CONDITIONAL CONVENTION LOADING

- `.claude/rules/` files with YAML frontmatter `paths` fields containing glob patterns
- Path-scoped rules load only when editing matching files → reduces irrelevant context and token usage
- Advantage over directory CLAUDE.md: glob patterns work for files spread across multiple directories

EXAMPLE

`paths: ["**/*.test.tsx"]` applies testing conventions to all test files regardless of location, while directory-level CLAUDE.md would be missed for tests in other directories.

3.4 — PLAN MODE VS DIRECT EXECUTION

USE PLAN MODE WHEN

- Complex tasks with large-scale changes (library migrations across 45+ files)
- Multiple valid approaches / architectural decisions
- Multi-file modifications with unknown dependencies
- Microservice restructuring, choosing between integration approaches

USE DIRECT EXECUTION WHEN

- Simple, well-scoped changes (single-file bug fix with clear stack trace)
- Adding a single validation check to one function

Explore subagent: Use for verbose discovery phases to prevent context window exhaustion during multi-phase tasks.

3.5 — ITERATIVE REFINEMENT TECHNIQUES

- Concrete input/output examples > prose descriptions for communicating expected transformations
- Test-driven iteration: write test suites first, iterate by sharing test failures
- Interview pattern: have Claude ask questions before implementing (surfaces cache invalidation strategies, failure modes)
- Provide all interacting issues in single message; fix independent issues sequentially

3.6 — INTEGRATE CLAUDE CODE INTO CI/CD

KNOW

- `-p` (or `--print`) flag → non-interactive mode for automated pipelines
- `--output-format json` and `--json-schema` CLI flags → structured output in CI contexts
- CLAUDE.md provides project context (testing standards, fixture conventions, review criteria) to CI-invoked Claude Code
- **Session context isolation:** the Claude session that generated code is less effective at reviewing its own changes vs an independent review instance

DO

- Run Claude Code in CI with `-p` flag to prevent interactive input hangs
- Include prior review findings in context when re-running after new commits (report only new/still-unaddressed issues)
- Provide existing test files in context so test generation avoids duplicate scenarios

Domain 4: Prompt Engineering & Structured Output

4

Prompt Engineering & Structured Output

20%

4.1 — EXPLICIT CRITERIA TO REDUCE FALSE POSITIVES

- Explicit criteria > vague instructions: "flag comments only when claimed behavior contradicts actual code" > "check that comments are accurate"
- General instructions like "be conservative" or "only report high-confidence findings" fail to improve precision
- High false positive rates on specific categories undermine confidence in accurate categories

TRAP ANSWER

"Add confidence scoring" is often wrong as a first step. The right first step for false positives is adding *explicit categorical criteria* — what to flag vs what to skip.

4.2 — FEW-SHOT PROMPTING FOR CONSISTENCY

- Few-shot examples: most effective technique for consistently formatted, actionable output when detailed instructions alone produce inconsistent results

- 2-4 targeted examples for ambiguous scenarios — show reasoning for why one action was chosen over plausible alternatives
- Use for: demonstrating ambiguous-case handling, format consistency, hallucination reduction in extraction
- Few-shot enables model to generalize judgment to novel patterns (not just match pre-specified cases)

4.3 — STRUCTURED OUTPUT VIA TOOL USE AND JSON SCHEMAS

KNOW

- `tool_use` with JSON schemas = most reliable approach for guaranteed schema-compliant output
- Strict schemas eliminate JSON **syntax** errors but NOT **semantic** errors (line items that don't sum to total, values in wrong fields)
- Schema design: required vs optional fields; enum with `"other"` + detail string for extensible categories
- Make fields nullable when source docs may not contain the info → prevents model from fabricating values

DO

- Set `tool_choice: "any"` to guarantee structured output when doc type is unknown
- Force specific tool first with `tool_choice: {"type": "tool", "name": "extract_metadata"}` before enrichment steps
- Add enum values like `"unclear"` and `"other"` + detail fields for ambiguous/extensible categorization

4.4 — VALIDATION, RETRY, AND FEEDBACK LOOPS

- Retry-with-error-feedback: append specific validation errors to prompt on retry to guide self-correction
- Retries are **ineffective** when required information is simply absent from source document
- `detected_pattern` field: track which code constructs trigger findings → enables systematic dismissal analysis
- Semantic validation errors vs schema syntax errors (the latter eliminated by `tool_use`)

4.5 — BATCH PROCESSING STRATEGIES (MESSAGE BATCHES API)

KNOW

- 50% cost savings, up to 24-hour processing window, **no guaranteed latency SLA**

- Batch API does NOT support multi-turn tool calling within a single request
- `custom_id` fields for correlating batch request/response pairs

APPROPRIATE FOR

- Overnight reports, weekly audits, nightly test generation (latency-tolerant)

NOT APPROPRIATE FOR

- Blocking pre-merge checks, any workflow where developers wait for results

4.6 — MULTI-INSTANCE AND MULTI-PASS REVIEW ARCHITECTURES

- Self-review limitation: model retains reasoning context from generation → less likely to question own decisions
- Independent review instances (without prior reasoning context) more effective at catching subtle issues
- Multi-pass: per-file local analysis passes + separate cross-file integration pass
- Run verification passes where model self-reports confidence alongside each finding → enables review routing

Domain 5: Context Management & Reliability

5

Context Management & Reliability

15%

5.1 — PRESERVE CRITICAL INFORMATION ACROSS LONG INTERACTIONS

- Progressive summarization risks: condensing numerical values, percentages, dates into vague summaries
- **"Lost in the middle" effect:** models reliably process info at beginning and end of long inputs; may omit findings from middle sections
- Tool results accumulate disproportionately (40+ fields per order lookup when only 5 are relevant)

DO

- Extract transactional facts into persistent "case facts" block included in each prompt, outside summarized history
- Trim verbose tool outputs to only relevant fields before they accumulate
- Place key findings summaries at **beginning** of aggregated inputs; organize details with section headers

5.2 — ESCALATION AND AMBIGUITY RESOLUTION PATTERNS

KNOW

- Escalation triggers: (1) customer explicitly requests human, (2) policy exceptions/gaps, (3) inability to make meaningful progress
- When customer explicitly demands human → escalate immediately, do NOT attempt investigation first
- Sentiment-based escalation and self-reported confidence scores = unreliable proxies for case complexity

TRAP

Using sentiment analysis or LLM confidence scores to trigger escalation is a distractor. The exam expects explicit criteria with few-shot examples of when to escalate vs. resolve.

5.3 — ERROR PROPAGATION ACROSS MULTI-AGENT SYSTEMS

- Structured error context enables intelligent coordinator recovery: failure type, attempted query, partial results, alternative approaches
- Distinguish **access failures** (need retry) from **valid empty results** (successful query, no matches)
- Subagents: handle local recovery for transient failures; propagate only unresolvable errors

ANTI-PATTERNS

Silently suppressing errors (returning empty results as success) · Terminating entire workflow on single subagent failure

5.4 — CONTEXT MANAGEMENT IN LARGE CODEBASE EXPLORATION

- Context degradation: models start giving inconsistent answers, referencing "typical patterns" rather than specific classes discovered earlier
- Scratchpad files for persisting key findings across context boundaries
- Structured state persistence for crash recovery: agents export state to known location; coordinator loads manifest on resume
- `/compact` to reduce context usage during extended exploration

5.5 — HUMAN REVIEW WORKFLOWS AND CONFIDENCE CALIBRATION

- Aggregate accuracy metrics (e.g., 97% overall) may mask poor performance on specific document types

- Stratified random sampling for measuring error rates in high-confidence extractions
- Field-level confidence scores calibrated using labeled validation sets for routing review attention
- Validate by document type AND field before automating high-confidence extractions

5.6 — INFORMATION PROVENANCE IN MULTI-SOURCE SYNTHESIS

- Source attribution is lost during summarization when findings are compressed without claim-source mappings
- Conflicting statistics from credible sources → annotate conflicts with source attribution, do NOT arbitrarily select one
- Temporal data: require publication/collection dates in structured outputs to prevent misinterpretation as contradictions
- Require subagents to output structured claim-source mappings (URLs, doc names, excerpts) preserved through synthesis

12 Official Sample Questions with Answers

These are directly from the official exam guide. Master the reasoning, not just the answers.

SCENARIO 1 — CUSTOMER SUPPORT RESOLUTION AGENT

Q1: Production data shows that in 12% of cases, your agent skips `get_customer` entirely and calls `lookup_order` using only the customer's stated name, occasionally leading to misidentified accounts and incorrect refunds. What change would most effectively address this?

✓ A) Add a programmatic prerequisite that blocks `lookup_order` and `process_refund` calls until `get_customer` has returned a verified customer ID

× B) Enhance the system prompt to state that `get_customer` is mandatory before any order operations

× C) Add few-shot examples showing the agent always calling `get_customer` first

× D) Implement a routing classifier that analyzes each request and enables only appropriate tools

Why A: When critical business logic (financial consequences) requires deterministic ordering, programmatic enforcement provides guarantees that prompt-based approaches cannot. B and C rely on probabilistic LLM compliance. D addresses tool availability, not tool ordering.

SCENARIO 1 — CUSTOMER SUPPORT RESOLUTION AGENT

Q2: Production logs show the agent frequently calls `get_customer` when users ask about orders (e.g., "check my order #12345"), instead of `lookup_order`. Both tools have minimal descriptions ("Retrieves customer information" / "Retrieves order details"). What's the most effective first step?

× A) Add few-shot examples (5-8) showing order-related queries routing to `lookup_order`

✓ B) Expand each tool's description to include input formats it handles, example queries, edge cases, and boundaries explaining when to use it vs similar tools

× C) Implement a routing layer that parses user input before each turn and pre-selects the appropriate tool

× D) Consolidate both tools into a single `lookup_entity` tool

Why B: Tool descriptions are the primary mechanism LLMs use for tool selection. Minimal descriptions cause the root problem. B is a low-effort, high-leverage fix. A adds token overhead without fixing the underlying issue. C is over-engineered and bypasses the LLM's natural language understanding.

SCENARIO 1 — CUSTOMER SUPPORT RESOLUTION AGENT

Q3: Your agent achieves 55% first-contact resolution vs the 80% target. Logs show it escalates straightforward cases while autonomously handling complex situations requiring policy exceptions. What's the most effective way to improve escalation calibration?

✓ A) Add explicit escalation criteria to your system prompt with few-shot examples demonstrating when to escalate vs resolve autonomously

✗ B) Have the agent self-report a confidence score (1-10) before each response and route to humans when confidence falls below a threshold

✗ C) Deploy a separate classifier model trained on historical tickets

✗ D) Implement sentiment analysis to detect frustration and auto-escalate on negative sentiment

Why A: Explicit escalation criteria with few-shot examples directly addresses unclear decision boundaries. B fails because LLM self-reported confidence is poorly calibrated. C requires labeled data and ML infrastructure when prompt optimization hasn't been tried. D: sentiment doesn't correlate with case complexity.

SCENARIO 2 — CODE GENERATION WITH CLAUDE CODE

Q4: You want to create a custom `/review` slash command that runs your team's standard code review checklist. This command should be available to every developer when they clone the repository. Where should you create this command file?

✓ A) In the `.claude/commands/` directory in the project repository

✗ B) In `~/.claude/commands/` in each developer's home directory

✗ C) In the `CLAUDE.md` file at the project root

✗ D) In a `.claude/config.json` file with a `commands` array

Why A: Project-scoped commands in `.claude/commands/` are version-controlled and automatically available to all developers who clone. B is personal-only. C is for instructions/context, not command definitions. D doesn't exist.

SCENARIO 2 — CODE GENERATION WITH CLAUDE CODE

Q5: You've been assigned to restructure a monolithic application into microservices — changes across dozens of files, decisions about service boundaries and module

dependencies. Which approach?

✓ A) Enter plan mode to explore the codebase, understand dependencies, and design an implementation approach before making changes

× B) Start with direct execution and make changes incrementally, letting the implementation reveal service boundaries

× C) Use direct execution with comprehensive upfront instructions detailing how each service should be structured

× D) Begin in direct execution and only switch to plan mode if you encounter unexpected complexity

Why A: Plan mode is designed for large-scale changes, multiple valid approaches, and architectural decisions — exactly what microservices restructuring requires. B risks costly rework when dependencies are discovered late. The complexity is already known upfront (not something that "might emerge").

SCENARIO 2 — CODE GENERATION WITH CLAUDE CODE

Q6: Your codebase has distinct areas with different conventions. Test files are spread throughout the codebase (e.g., `Button.test.tsx` next to `Button.tsx`), and you want all tests to follow the same conventions regardless of location. What's the most maintainable approach?

✓ A) Create rule files in `.claude/rules/` with YAML frontmatter specifying glob patterns to conditionally apply conventions based on file paths

× B) Consolidate all conventions in the root `CLAUDE.md`, relying on Claude to infer which section applies

× C) Create skills in `.claude/skills/` for each code type with relevant conventions

× D) Place a separate `CLAUDE.md` in each subdirectory containing that area's specific conventions

Why A: `.claude/rules/` with glob patterns (e.g., `**/*.test.tsx`) applies conventions to all test files regardless of directory location. B relies on inference rather than explicit

matching. C requires manual skill invocation. D can't handle files spread across many directories since CLAUDE.md files are directory-bound.

SCENARIO 3 — MULTI-AGENT RESEARCH SYSTEM

Q7: After running the system on "impact of AI on creative industries," the final reports cover only visual arts — missing music, writing, and film production. Coordinator logs show it decomposed into: "AI in digital art creation," "AI in graphic design," and "AI in photography." Root cause?

× A) The synthesis agent lacks instructions for identifying coverage gaps

✓ B) The coordinator agent's task decomposition is too narrow, resulting in subagent assignments that don't cover all relevant domains

× C) The web search agent's queries aren't comprehensive enough

× D) The document analysis agent is filtering out non-visual sources

Why B: The coordinator's logs reveal the root cause directly — it decomposed "creative industries" into only visual arts subtopics. The subagents executed their assigned tasks correctly. The problem is what they were assigned. Options A, C, D incorrectly blame downstream agents that are working within their scope.

SCENARIO 3 — MULTI-AGENT RESEARCH SYSTEM

Q8: The web search subagent times out while researching a complex topic. You need to design how this failure information flows back to the coordinator. Which error propagation approach best enables intelligent recovery?

✓ A) Return structured error context to the coordinator including failure type, attempted query, any partial results, and potential alternative approaches

× B) Implement automatic retry with exponential backoff, returning generic "search unavailable" only after retries exhausted

× C) Catch timeout in the subagent and return an empty result set marked as successful

× D) Propagate the timeout exception directly to a top-level handler that terminates the entire research workflow

Why A: Structured error context lets the coordinator make intelligent decisions — retry with modified query, try alternative approach, or proceed with partial results. B hides context from the coordinator. C suppresses the error (success + empty = no recovery). D terminates unnecessarily when recovery could succeed.

SCENARIO 3 — MULTI-AGENT RESEARCH SYSTEM

Q9: The synthesis agent frequently returns to coordinator to verify specific claims, adding 2-3 round trips per task (+40% latency). 85% of verifications are simple fact-checks; 15% require deeper investigation. Most effective approach to reduce overhead while maintaining reliability?

✓ A) Give the synthesis agent a scoped `verify_fact` tool for simple lookups, while complex verifications continue delegating to the web search agent through the coordinator

× B) Have the synthesis agent accumulate all verification needs and return them as a batch to the coordinator at end of its pass

× C) Give the synthesis agent access to all web search tools so it can handle any verification need directly

× D) Have the web search agent proactively cache extra context anticipating what synthesis will need to verify

Why A: Least privilege — give synthesis agent only what it needs for the 85% common case while preserving coordination for complex cases. B creates blocking dependencies (synthesis steps may depend on verified facts). C over-provisions the synthesis agent. D relies on speculative caching that can't reliably predict needs.

SCENARIO 5 — CLAUDE CODE FOR CONTINUOUS INTEGRATION

Q10: Your pipeline script runs `claude "Analyze this pull request for security issues"` but the job hangs indefinitely. Logs indicate Claude Code is waiting for interactive input. What's the correct approach?

✓ A) Add the `-p` flag: `claude -p "Analyze this pull request for security issues"`

× B) Set environment variable `CLAUDE_HEADLESS=true` before running

× C) Redirect stdin: `claude "Analyze this pull request..." < /dev/null`

× D) Add the `--batch` flag

Why A: The `-p` (`--print`) flag is the documented way to run Claude Code in non-interactive mode. It processes the prompt, outputs to stdout, and exits without waiting for user input. B, C, D reference non-existent features or Unix workarounds that don't properly address Claude Code's command syntax.

SCENARIO 5 — CLAUDE CODE FOR CONTINUOUS INTEGRATION

Q11: Two workflows use real-time Claude calls: (1) blocking pre-merge check that must complete before developers can merge, and (2) overnight technical debt report. Manager proposes switching both to Message Batches API for 50% cost savings. How should you evaluate?

✓ A) Use batch processing for the technical debt reports only; keep real-time calls for pre-merge checks

× B) Switch both workflows to batch processing with status polling to check for completion

× C) Keep real-time calls for both workflows to avoid batch result ordering issues

× D) Switch both to batch processing with a timeout fallback to real-time if batches take too long

Why A: Batch API offers 50% savings but has up to 24-hour processing with no guaranteed SLA — unsuitable for blocking pre-merge checks where developers wait. Perfect for latency-tolerant overnight reports. B/D would block developers unacceptably. C misses an opportunity to save costs on latency-tolerant workloads.

SCENARIO 5 — CLAUDE CODE FOR CONTINUOUS INTEGRATION

Q12: Your single-pass review analyzing 14 files together produces inconsistent results: detailed feedback for some files, superficial for others, bugs missed, contradictory feedback within the same PR. How should you restructure?

✓ A) Split into focused passes: analyze each file individually for local issues, then run a separate integration-focused pass examining cross-file data flow

× B) Require developers to split large PRs into 3-4 file submissions before the automated review runs

× C) Switch to a higher-tier model with a larger context window

× D) Run three independent review passes on the full PR and only flag issues appearing in at least two of three runs

Why A: Splitting reviews into focused passes directly addresses attention dilution when processing many files at once. File-by-file ensures consistent depth; integration pass catches cross-file issues. B shifts burden to developers. C misunderstands that larger context doesn't solve attention quality. D would suppress intermittent-but-real bugs.

4 Hands-On Preparation Exercises

Build these before the exam. Hands-on experience is how the exam was designed to be studied for.

Exercise 1: Multi-Tool Agent with Escalation Logic

Goal: Practice agentic loops, structured error handling, escalation patterns.

1. Define 3-4 MCP tools with detailed descriptions that clearly differentiate purpose, inputs, and boundary conditions. Include at least two tools with similar functionality.
2. Implement an agentic loop that checks `stop_reason` — continue on `"tool_use"`, terminate on `"end_turn"`.
3. Add structured error responses: `errorCategory` (transient/validation/permission), `isRetryable` boolean, human-readable descriptions. Test each error type.

4. Implement a programmatic hook that intercepts tool calls to enforce a business rule (e.g., block operations above a threshold), redirecting to escalation workflow.
5. Test with multi-concern messages and verify the agent decomposes, handles each concern, synthesizes a unified response.

REINFORCES: DOMAIN 1 (AGENTIC ARCHITECTURE), DOMAIN 2 (TOOL DESIGN), DOMAIN 5 (RELIABILITY)

Exercise 2: Configure Claude Code for a Team Development Workflow

Goal: Practice CLAUDE.md hierarchies, custom slash commands, path-specific rules, MCP integration.

1. Create a project-level CLAUDE.md with universal coding standards. Verify instructions are consistently applied.
2. Create `.claude/rules/` files with YAML frontmatter glob patterns: `paths: ["src/api/**/*"]` for API conventions, `paths: ["**/*.test.*"]` for testing conventions. Test that rules load only when editing matching files.
3. Create a project-scoped skill with `context: fork` and `allowed-tools` restrictions. Verify it runs in isolation without polluting main conversation.
4. Configure a shared MCP server in `.mcp.json` with environment variable expansion. Add a personal experimental MCP server in `~/.claude.json`. Verify both are simultaneously available.
5. Test plan mode vs direct execution on: a single-file bug fix, a multi-file library migration, and a new feature with multiple valid approaches.

REINFORCES: DOMAIN 3 (CLAUDE CODE), DOMAIN 2 (TOOL DESIGN)

Exercise 3: Structured Data Extraction Pipeline

Goal: Practice JSON schemas, `tool_use` for structured output, validation-retry loops, batch processing.

1. Define an extraction tool with a JSON schema containing required and optional fields, an enum with an `"other"` + detail string pattern, and nullable fields for absent information. Verify the model returns null rather than fabricating values.
2. Implement a validation-retry loop: when Pydantic/JSON schema validation fails, send a follow-up request including the document, failed extraction, and specific validation error. Track which errors are resolvable via retry vs not.

3. Add few-shot examples demonstrating extraction from documents with varied formats (inline citations vs bibliographies, narrative vs structured tables). Verify improved structural variety handling.
4. Design a batch processing strategy: submit 100 documents using Message Batches API, handle failures by `custom_id`, resubmit failed documents with modifications, calculate total processing time relative to SLA constraints.
5. Implement human review routing: model outputs field-level confidence scores → route low-confidence extractions to human review → analyze accuracy by document type and field.

REINFORCES: DOMAIN 4 (PROMPT ENGINEERING), DOMAIN 5 (CONTEXT MANAGEMENT)

Exercise 4: Multi-Agent Research Pipeline

Goal: Practice subagent orchestration, context passing, error propagation, synthesis with provenance tracking.

1. Build a coordinator agent delegating to at least two subagents (web search + document analysis). Ensure `allowedTools` includes `"Task"` and each subagent receives findings directly in its prompt (no automatic context inheritance).
2. Implement parallel subagent execution by having the coordinator emit multiple `Task` calls in a single response. Measure latency improvement vs sequential execution.
3. Design structured output for subagents separating content from metadata: each finding includes claim, evidence excerpt, source URL/document name, and publication date. Verify synthesis agent preserves source attribution when combining findings.
4. Implement error propagation: simulate a subagent timeout and verify the coordinator receives structured error context (failure type, attempted query, partial results). Test that coordinator can proceed with partial results and annotate the output with coverage gaps.
5. Test with conflicting source data (two credible sources with different statistics) and verify the synthesis output preserves both values with source attribution rather than arbitrarily selecting one.

REINFORCES: DOMAIN 1 (AGENTIC ARCHITECTURE), DOMAIN 2 (TOOL DESIGN), DOMAIN 5 (RELIABILITY)

Key Terms & Technology Reference

stop_reason "tool_use" = continue loop; "end_turn" = present response	Task tool Mechanism for spawning subagents; must be in allowedTools	PostToolUse Hook that intercepts tool results before model processes them
fork_session Creates independent branches from a shared analysis baseline	--resume Named session resumption: --resume <session-name>	tool_choice "auto" / "any" / forced { "type": "tool", "name": "..."}
isError MCP flag for communicating tool failures to the agent	isRetryable Boolean in error response indicating if retry may succeed	errorCategory transient / validation / business / permission
custom_id Correlates batch request/response pairs in Message Batches API	-p / --print Claude Code non-interactive mode for CI/CD pipelines	--output-format json Structured output flag for CI contexts
context: fork Skill frontmatter; isolates skill output from main conversation	allowed-tools Skill frontmatter; restricts tool access during skill execution	argument-hint Skill frontmatter; prompts developers for required parameters
@import CLAUDE.md syntax for referencing external modular files	.claude/rules/ Topic-specific rule files with YAML frontmatter path scoping	/compact Reduces context usage during extended exploration sessions
detected_pattern Field for tracking code constructs that trigger findings	Message Batches API 50% savings, 24hr window, no latency SLA, no multi-turn tool calling	

In-Scope vs Out-of-Scope

✓ IN SCOPE — WILL BE TESTED

- Agentic loop implementation (stop_reason control flow)
- Multi-agent orchestration, parallel subagent execution
- Subagent context management, structured state persistence
- Tool interface design and description quality
- MCP tool/resource design and server configuration
- Error handling and propagation (structured errors)
- Escalation decision-making with explicit criteria
- CLAUDE.md configuration hierarchy
- Custom commands and skills with frontmatter
- Plan mode vs direct execution decisions
- Iterative refinement (few-shot, test-driven, interview pattern)
- Structured output via tool_use + JSON schemas
- Batch processing appropriateness and latency tolerance
- Context window optimization techniques
- Human review workflows and confidence calibration
- Information provenance in multi-source synthesis

✗ OUT OF SCOPE — WILL NOT BE TESTED

- Fine-tuning Claude models or training custom models
- Claude API authentication, billing, or account management
- Specific programming language/framework implementation details
- Deploying or hosting MCP servers (infrastructure, networking)
- Claude's internal architecture, training process, or model weights
- Constitutional AI, RLHF, or safety training methodologies
- Embedding models or vector database implementation details
- Computer use (browser automation, desktop interaction)
- Vision/image analysis capabilities
- Streaming API or server-sent events implementation
- Rate limiting, quotas, or API pricing calculations
- OAuth, API key rotation, authentication protocol details
- Specific cloud provider configurations (AWS, GCP, Azure)
- Performance benchmarking or model comparison metrics
- Prompt caching implementation details (beyond knowing it exists)
- Token counting algorithms or tokenization specifics

Study Checklist

Check off as you complete each preparation step from the official exam guide.

- ☐ Build a complete agentic loop with tool calling, error handling, and session management. Practice spawning subagents and passing context between them.

- ☐ Configure Claude `.claude/rules/`, custom `context: fork`, `allowed-tools`, and Code for a real project: skills with frontmatter integrate CLAUDE.md hierarchy, at least path-specific rules in (one MCP server.

- ☐ Design and test MCP tools: write tool descriptions that clearly differentiate similar tools. Implement structured error responses with error categories and retryable flags. Test tool selection reliability with ambiguous requests.

- ☐ Build a structured `tool_use` with JSON schemas, implement validation-retry loops, design data extraction schemas with optional/nullable fields, practice batch pipeline: use processing with the Message Batches API.

- ☐ Practice prompt engineering techniques: write few-shot examples for ambiguous scenarios. Define explicit review criteria to reduce false positives. Design multi-pass review architectures for large code reviews.

- ☐ Study context management patterns: practice extracting structured facts from verbose tool outputs, implementing scratchpad files for long sessions, designing subagent delegation to manage context limits.

- ☐ Review escalation and human-in-the-loop patterns: know when to escalate (policy gaps, customer requests, inability to progress) vs resolve autonomously. Practice designing human review workflows with confidence-based routing.

- ☐ Complete the practice exam (linked separately from the exam guide) — covers same scenarios and question format, shows explanations after each answer.

Study guide built from the official Anthropic CCA-F Exam Guide (v0.1) · Generated April 2026

Register via the Claude Partner Network (free) · anthropic.skilljar.com for free prep courses